Managing Software Cost

CSE 362

Software Development and Professional Practice

Winter 2023

© Royal Military College of Canada

- Software development takes times and money.
- Hard to reliably estimate at an early stage.
- What does it really mean for a piece of software to be 50% complete?
- What are ways we can estimate software cost/schedule?

Cost vs. Effort?

- In most cost estimation models, a simple relation between cost and effort is assumed.
- The effort may be measured in man-months, for instance, and each man-month is taken to incur a fixed amount, say, of $5000.
- The total estimated cost is then obtained by simply multiplying the estimated number of man-months by this constant factor.
- Cost and effort may be used interchangeably.

What is included in the cost?

- The notion of total cost is usually taken to indicate the cost of the initial software development effort

the cost of the requirements engineering + design + implementation + testing phases.

- Maintenance costs are usually not taken into account.
- The notion of cost usually does not include possible hardware costs either: It concerns only personnel costs involved in software development.

A simple model to estimate cost

$$E = 2.7\ KLOC^{1.05}$$

- KLOC = Kilo Lines of Code = Lines of Code / 1000
- The effort measure could be the number of man-months needed.

What is a line of code?

Do we count machine code, or the source code in some high-level language?

Do we count comment lines, or blank lines that increase readability?

Do we take into account holidays, sick-leave, and the like, in our notion of the man-month, or does it concern a net measure?

How to determine the equations of an algorithmic cost estimation model?

- Perform experiments

- Analysis of real-world projects

Algorithmic Models for Cost estimation

- Programming effort is strongly correlated with program size.
- There exist various models that express this correlation.

Origin	Base formula
Halstead	$E = 0.7KLOC^{1.50}$
Boehm	$E = 2.4KLOC^{1.05}$
Walston--Felix	$E = 5.2KLOC^{0.91}$

Algorithmic Models for Cost estimation

$KLOC$	$E = 0.7KLOC^{1.50}$	$E = 2.4KLOC^{1.05}$	$E = 5.2KLOC^{0.91}$
1	0.7	2.4	5.2
10	22.1	26.9	42.3
50	247.5	145.9	182.8
100	700.0	302.1	343.6
1000	22135.9	3390.1	2792.6

How to get a *reliable* estimate of the software size early on

- How should we estimate the number of pages in a novel not yet written?

- The further advanced we are with the project, the more accurate our size estimate will get.
- If the design is more or less finished, we may (possibly) form a reasonable impression of the size of the resulting software.

- If lines of code cannot be estimated fairly decently, we have to look for an alternative (more on that in a bit).

COCOMO

$$E = b\ KLOC^{c}$$

- *b* and *c* are constants that depend on *the kind of project* that is being executed. COCOMO distinguishes three classes of project:

- Organic: small team – known environment – experience developers
  - b = 2.4, c = 1.05
- Embedded: product embedded in environment that is inflexible, poses severe constraints
  - b = 3.6, c = 1.20
- Semidetached: intermediate form
  - b = 3.0, c = 1. 12

COCOMO

KLOC	Effort in man-months organic ($E = 2.4KLOC^{1.05}$)	semidetached ($E = 3.0KLOC^{1.12}$)	embedded ($E = 3.6KLOC^{1.20}$)
1	2.4	3.0	3.6
10	26.9	39.6	57.1
50	145.9	239.4	392.9
100	302.1	521.3	904.2
1000	3390.0	6872.0	14333.0

Size versus effort in Basic COCOMO

Function Point Analysis (FPA)

- a method of estimating costs where the problems associated with determining the expected amount of code can be avoided.

- FPA is based on counting the number of different data structures that are used.

- FPA is particularly suitable for projects aimed at realizing business applications where the structure of the data plays a very dominant role.

Function Point Analysis (FPA)

The following five entities play a central role in the FPA-model:

- Number of input types (I)
- Number of output types (O)
- Number of inquiry types (E)
- Number of logical internal files (L)
- Number of interfaces (F)

- The # of Unadjusted Function points:

$$UFP = 4I + 5O + 4E + 10L + 7F$$

Function Point Analysis (FPA)

- Then, a direct mapping from (adjusted) function points to lines of code is defined

- Example: one function point corresponds to 65 lines of PL/I, or to 100 lines of COBOL

Next…

Next topic